Laboratorium nr 3 – wyszukiwanie pełnotekstowe: SQL kontra Solr

Na wykładach 4 i 5 porównaliśmy trzy sposoby przeszukiwania tekstu: LIKE, indeks pełnotekstowy w bazie danych (SQL Server, PostgreSQL) i dedykowany silnik wyszukiwania Apache Solr. Na zbiorze 35 tysięcy filmów wszystkie odpowiadały w ułamku sekundy — prawdziwe różnice widać dopiero przy dużych danych i przy funkcjach, których baza danych po prostu nie ma.

Znajdziecie albo wygenerujecie własny zbiór danych tekstowych, zaindeksujecie go w bazie i w Solrze, a następnie porównacie oba podejścia — funkcjonalnie i wydajnościowo.

Zasada jest prosta: **im większy zbiór, tym lepiej**. Przy kilkudziesięciu tysiącach rekordów nawet `LIKE '%słowo%'` wydaje się wystarczający; przy kilku milionach już nie.

Organizacja pracy

Pracujecie w parach. Poziom trudności (3.0 / 4.0 / 5.0) para wybiera wspólnie.

- część SQL realizujecie w wybranym silniku: **SQL Server** albo **PostgreSQL** — para wybiera jeden;
- część Solr realizujecie w Apache Solr 10 (Docker albo paczka zip);
- język programowania loadera i klienta jest dowolny — punktem startu może być solucja SolrMovies z wykładu 5 (C#, HttpClient).

Poziomy są **narastające**: na 4.0 obowiązuje wszystko z poziomu 3.0 oraz wymagania dodatkowe, na 5.0 — wszystko z poziomu 4.0 oraz kolejne wymagania.

Poziomy 4.0 i 5.0 zawierają listy wyboru: realizujecie wskazaną liczbę pozycji, a nie całą listę. Każda lista kończy się punktem otwartym — jeśli macie własny pomysł o zbliżonym poziomie trudności, śmiało go zrealizujcie.

Poziomy w skrócie

Poziom	Zbiór danych	Część SQL	Część Solr	Oddajecie
3.0	min. 100 000 rekordów	indeks pełnotekstowy, 3 zapytania + LIKE	własny schemat, indeksowanie, te same 3 zapytania	kod + README z tabelą wyników
4.0	min. 500 000 rekordów	+ ranking, GROUP BY	+ eDisMax, facety, podświetlanie, pomiary	+ sprawozdanie PDF, odpowiedź ustna
5.0	jak na 4.0	+ agregacje vs facety	+ skalowanie, facet zakresowy	+ sprawozdanie PDF, odpowiedź ustna

Zbiór danych (wszystkie poziomy)

Wymagania

- co najmniej jedno pole z tekstem w języku naturalnym, średnio min. 300 znaków na rekord (opis, recenzja, treść posta, abstrakt, fabuła, artykuł);
- co najmniej dwa pola kategoryczne o ograniczonej liczbie wartości (kategoria, kraj, autor, tag) — posłużą do filtrów i facetów;
- co najmniej jedno pole liczbowe albo data (rok, cena, ocena, data publikacji) — do zakresów i sortowania;
- unikalny klucz;
- liczba rekordów zgodna z wybranym poziomem — im więcej, tym lepiej.

Skąd wziąć dane

- gotowe zbiory: Kaggle, Hugging Face Datasets, Stack Exchange Data Dump, zrzuty Wikipedii, metadane i abstrakty arXiv, recenzje produktów, filmów lub książek, otwarte dane publiczne (np. dane.gov.pl);
- dane syntetyczne: generator w Pythonie (Faker) albo w C# (Bogus), ewentualnie z pomocą LLM. Teksty muszą przypominać język naturalny — prawdziwe słowa, odmiana, różna długość. „Lorem ipsum" odpada: na nim stemming, stoplisty i ranking nie mają czego pokazać;
- powielanie danych (jak na wykładach: `CROSS JOIN` w SQL, `--copies 10` w Solrze) jest dozwolone, żeby zwiększyć wolumen, ale kopie muszą być oznaczone (np. pole `copy_no`). W sprawozdaniu opiszcie, jak powielenie wpłynęło na wyniki (duplikaty w top-10, liczności facetów);
- podajcie źródło i licencję zbioru. Nie używamy prawdziwych danych osobowych.

Zadanie na 3.0

Część SQL (SQL Server albo PostgreSQL)

- import zbioru do tabeli z kluczem głównym (`BULK INSERT`, `COPY` albo własny loader) — min. 100 000 rekordów;
- indeks pełnotekstowy na polu tekstowym: `FULLTEXT CATALOG` + `FULLTEXT INDEX` (SQL Server) albo kolumna `tsvector` z indeksem `GIN` (PostgreSQL);
- min. 3 zapytania pełnotekstowe:
  - pojedyncze słowo w dowolnej formie (np. „run" znajduje „running");
  - fraza — słowa obok siebie, w podanej kolejności;
  - logika z negacją — np. słowo A oraz (B lub C), bez D;
- zapytanie nr 1 wykonane dodatkowo przez `LIKE '%…%'` — dla porównania czasu i liczby wyników.

Część Solr

- Solr 10 uruchomiony w Dockerze albo z paczki zip;
- kolekcja z własnym schematem i wyłączonym trybem schemaless (`update.autoCreateFields=false`); minimum: klucz, analizowane pole tekstowe (typ odpowiedni dla języka danych, np. `text_en`), dwa pola `string`, pole liczbowe albo data;
- zaindeksowanie całego zbioru własnym programem przez `/update` — paczki dokumentów, jeden commit na końcu;
- te same 3 zapytania w Solrze, z użyciem parametrów `q`, `fq`, `fl`, `sort`, `rows`; każde zapytanie także jako URL wklejony w przeglądarkę lub wpisany w Admin UI → Query.

Wyniki

- tabela: zapytanie | liczba wyników w SQL | numFound w Solrze | czas w SQL | QTime w Solrze — czasy z drugiego uruchomienia;
- 2–3 zdania komentarza: skąd różnice w liczbie wyników (stoplisty, stemming, tokenizacja).

Poziom 3.0 nie wymaga sprawozdania ani odpowiedzi ustnej — wystarczy README z instrukcją uruchomienia i tabelą wyników.

Zadanie na 4.0

Wymagania obowiązkowe

- wszystko z poziomu 3.0;
- zbiór min. 500 000 rekordów;

- SQL: ranking — `CONTAINSTABLE` / `FREETEXTTABLE` z kolumną `RANK` (SQL Server) albo `ts_rank_cd` (PostgreSQL) — top-10 dla 3 zapytań z poziomu 3.0;
- SQL: jedna agregacja `GROUP BY` po polu kategorycznym dla wyników zapytania pełnotekstowego;
- Solr: schemat z krótkim uzasadnieniem flag pól (`indexed` / `stored` / `docValues` / `multiValued`);
- Solr: te same 3 zapytania w eDisMax z `qf` (wagi pól) — top-10;
- Solr: facety dla dwóch pól kategorycznych (`facet.field`) oraz podświetlanie trafień (`hl`);
- porównanie top-10 z SQL i z Solra — ile dokumentów się powtarza i skąd różnice;
- pomiary: czas ładowania i indeksowania danych w obu systemach oraz czasy 3 zapytań (mediana z 5 uruchomień) — `LIKE`, indeks pełnotekstowy w SQL i Solr.

Wybierz co najmniej 1 rozszerzenie

- synonimy zarządzane przez REST w Solrze (min. 3 grupy synonimów, pokazany efekt na liczbie wyników);
- autouzupełnianie (edge n-gramy) — porównanie z `LIKE 'abc%'`;
- odporność na literówki: zapytania fuzzy (~) w Solrze kontra `LIKE` / `pg_trgm`;
- własny typ pola z jawnym łańcuchem analizy i zakładka Analysis dla 3 zdań — porównanie z `sys.dm_fts_parser` albo `to_tsvector`;
- More Like This — „podobne dokumenty” z oceną jakości na 3 przykładach;
- boost: łączenie trafności z polem liczbowym (np. nowsze lub lepiej oceniane wyżej);
- własne rozszerzenie o zbliżonym poziomie trudności.

Ponadto: sprawozdanie zgodnie z rozdziałem „Sprawozdanie” oraz odpowiedź ustna.

Zadanie na 5.0

Wymagania obowiązkowe

- wszystko z poziomu 4.0;
- skalowanie: czasy 3 zapytań dla trzech wielkości zbioru (np. 10%, 50% i 100% danych) — jeden wykres: `LIKE`, indeks pełnotekstowy w SQL i Solr, z krótkim wnioskiem, kiedy który sposób przestaje wystarczać;
- facety kontra `GROUP BY`: jedno zapytanie Solra z dwoma facetami i jednym facetem zakresowym (`facet.range`) wobec odpowiadających mu zapytań agregujących w SQL — wyniki i czasy.

Wybierz co najmniej 2 rozszerzenia

Z listy z poziomu 4.0 (inne niż zrealizowane wcześniej) albo z poniższych:

- duplikaty po powieleniu danych: `collapse` po identyfikatorze oryginału — jeden wynik na rekord;
- głębokie stronicowanie: `start`/`rows` kontra `cursorMark` w Solrze — pomiar dla dalekich stron;
- aktualizacja danych: zmiana i usunięcie kilku rekordów w bazie, przeniesienie zmian do Solra prostym skryptem i sprawdzenie, że wyniki w obu systemach się zgadzają;
- język polski: dane polskojęzyczne i analiza Stempel albo Morfologik (moduł analysis-extras);
- własne rozszerzenie o zbliżonym poziomie trudności, uzgodnione z prowadzącym.

Ponadto: sprawozdanie zgodnie z rozdziałem „Sprawozdanie” oraz odpowiedź ustna.

Metodyka pomiarów (wszystkie poziomy)

- te same dane w obu systemach: ta sama liczba rekordów i te same zapytania;
- czasy z drugiego i kolejnych uruchomień (rozgrzany cache); jeśli mierzycie „zimny” start, opiszcie, jak go uzyskaliście (SQL Server: `CHECKPOINT; DBCC DROPCLEANBUFFERS;`, PostgreSQL i Solr: restart usługi lub kontenera);

- przy każdym pomiarze zapisujecie dokładne zapytanie (SQL albo pełny URL Solra), liczbę wyników i czas;
- czas w SQL: `SET STATISTICS TIME ON` albo `EXPLAIN (ANALYZE, BUFFERS)`; czas w Solrze: `QTime` oraz czas zmierzony po stronie klienta;
- podajcie sprzęt i wersje: procesor, pamięć RAM, dysk, wersje bazy i Solra, pamięć przydzieloną Solrowi (`SOLR_HEAP`).

Sprawozdanie (poziomy 4.0 i 5.0)

Sprawozdanie w formacie PDF ma zawierać:

- opis zbioru danych: źródło albo sposób generowania, licencja, liczba rekordów, rozmiar, przykładowy rekord;
- schemat tabeli i indeksu pełnotekstowego w SQL oraz schemat Solra z uzasadnieniem flag pól i łańcucha analizy;
- wyniki porównania funkcjonalnego: zapytania, liczby wyników, rankingi, facety, podświetlanie;
- wyniki pomiarów w tabelach i na wykresach;
- opis wybranych rozszerzeń;
- dla poziomu 5.0 — wykres skalowania i porównanie facetów z GROUP BY;
- wnioski: kiedy indeks pełnotekstowy w bazie wystarcza, a kiedy warto sięgnąć po Solra — na podstawie Waszych liczb.

Oddanie pracy

Na platformie Delta wgrajcie jedno archiwum na parę, o nazwie:

LAB3_Nazwisko_Imie_Nazwisko_Imie.zip

Archiwum zawiera kod (generator albo skrypt pobierający dane, loader, zapytania), skrypty SQL, schemat Solra (JSON dla Schema API albo managed-schema), README z instrukcją uruchomienia, a na poziomach 4.0 i 5.0 także sprawozdanie PDF. Samego zbioru danych nie dołączajcie — wystarczy link do źródła albo generator.

Poziomy 4.0 i 5.0 wymagają dodatkowo odpowiedzi ustnej na laboratoriach.

Wskazówki techniczne

- Solr 10 w Dockerze — wystarczy jeden kontener:

```
docker run -d --name solr -p 8983:8983 -e SOLR_HEAP=2g solr:10
http://localhost:8983/solr          (Admin UI)
```

- punkt startu: solucja SolrMovies z wykładu 5 — tworzenie schematu (I01), loadery z SQL i CSV (I02, I03), zapytania i facety; plik zapytania.html pokazuje, jak wpisywać zapytania w przeglądarkę i w Admin UI;
- SQL Server: `BULK INSERT`, `CREATE FULLTEXT CATALOG` / `FULLTEXT INDEX`, `CONTAINS`, `FREETEXTTABLE`; PostgreSQL: `COPY`, `tsvector` + `GIN`, `websearch_to_tsquery`, `ts_rank_cd`, `ts_headline`, `pg_trgm`;
- generowanie danych: Python Faker, C# Bogus; duże pliki generujcie strumieniowo do CSV, a nie w pamięci;
- ładowanie do Solra: paczki 1 000–10 000 dokumentów w JSON, jeden commit na końcu;
- pamięć: Docker Desktop z min. 4–6 GB RAM; heap Solra 2–4 GB, resztę zostawcie systemowi na cache plików indeksu;
- dowolny język i biblioteki są dopuszczalne — liczy się poprawność porównania i jakość wniosków.